JARVIS-LV

Voice-controlled AI desktop assistant

Complete project documentation: details, workflow, working commands and limitations

Based on: Mark LV by FatihMakes (CC BY-NC 4.0), extended in this workspace with an offline assistant, a floating-robot chat UI, 21 extra tools and a self-improvement system.

Produced on 8 October 2026 by reading the source code in this repository and running its test files. Nothing here was observed with a live microphone or a live Gemini session; see section 11.

Contents

1. What JARVIS-LV is 3
2. Architecture 4
3. Workflow 5
4. Installing and running (terminal commands) 7
5. Voice commands that work 8
6. Memory, learning and data 11
7. Offline mode commands 13
8. Safety design 14
9. Model ladder (one-shot Gemini calls) 14
10. Project layout 15
11. Limitations and known issues 16
12. Quick troubleshooting 18
13. Portable installer (moving JARVIS to another PC) 18

1. What JARVIS-LV is

JARVIS-LV is a desktop voice assistant written in Python. You talk to it (or type); it answers out loud and can **do things on your computer**: open and close apps, change volume and brightness, manage files, browse the web, take screenshots, set timers, play Spotify, build PowerPoint decks and more. It looks at your screen or webcam when asked, remembers facts about you between sessions, and can undo many of its own actions.

The thinking is done by Google's **Gemini Live API**, which streams audio both ways and decides when to call one of the assistant's **tools** (Python functions that act on the machine). The application around it - audio capture, the user interface, the tools, the safety gates and the memory - runs locally. When the internet or Gemini is unavailable, a small rule-based **offline assistant** takes over.

Headline facts

Item	Value
Language / UI	Python 3.11 (developed on 3.11.9); PyQt6 desktop UI; asyncio core
Brain	Gemini Live (`gemini-3.1-flash-live-preview`, fallback `gemini-2.5-flash-native-audio-preview-12-2025`) plus a 9-model ladder for one-shot calls
Tools	**46** built in: 38 file-backed in `actions/` + 8 inline in `main.py` (counted by running the tool loader; 0 rejected). Plugins add more.
Platforms	Designed for Windows, macOS and Linux; this workspace is Windows 11 and several tools are Windows-only (section 11)
Voices	Charon (default), Puck, Kore, Fenrir, Aoede
Personalities	default, professional, casual, funny, grumpy, sarcastic, friendly, calm, hype
Source size	main.py 2,535 lines; ui.py 5,617; ui_chat.py 3,705; 38 action files
Licence	Creative Commons BY-NC 4.0 - personal, non-commercial use only

What is different from the upstream Mark LV

The README in the repository still describes upstream Mark LV. This workspace has uncommitted work on top of it (see `git status`): the web `dashboard/` was deleted, and these were added.

- **Offline assistant** (`core/offline_*.py`, `open_close_apps.py`): works with no internet, no AI model.
- **Floating robot + chat window** (`ui_chat.py`), now the default UI. The older HUD is still there (`"ui_mode": "classic"`).
- **Real confirmation gate** for irreversible actions, the model cannot press the button (`core/confirm.py`).
- **21 extra tools**: calculator, notes, timer, shell_command, process_manager, system_toggles, spotify, powerpoint, image_generate, logins (encrypted vault), clipboard_history, file_tools, network_tools, screenshot, daily_briefing, everyday, site_to_file, personality, lessons, routines, tool_builder.
- **Self-improvement system**: learns from corrections, notices routines, can write new tools for itself (all off by default except memory).
- **Tests** in `tests/` (four files, all headless).
- **Close button**: a round X in the chat window's top-right corner quits JARVIS and the robot (same as the robot's Quit menu); the dash button still hides it to the robot.
- **English start-up greeting**: the launch greeting and headline are always English; replies then follow the language you speak.
- **Robot icon**: `assets/jarvis_icon*.png` (transparent and rounded-tile versions) and `config/jarvis.ico` used for the installer, shortcuts and chat window.
- **Portable installer**: one `JARVIS-Setup.exe` that carries no personal data (section 13).

2. Architecture

Everything funnels through one class, `JarvisLive` in `main.py`. It owns the Gemini connection, the audio streams and the tool dispatcher. The UI is a separate layer that talks to it through callbacks, so the same core runs with either interface.

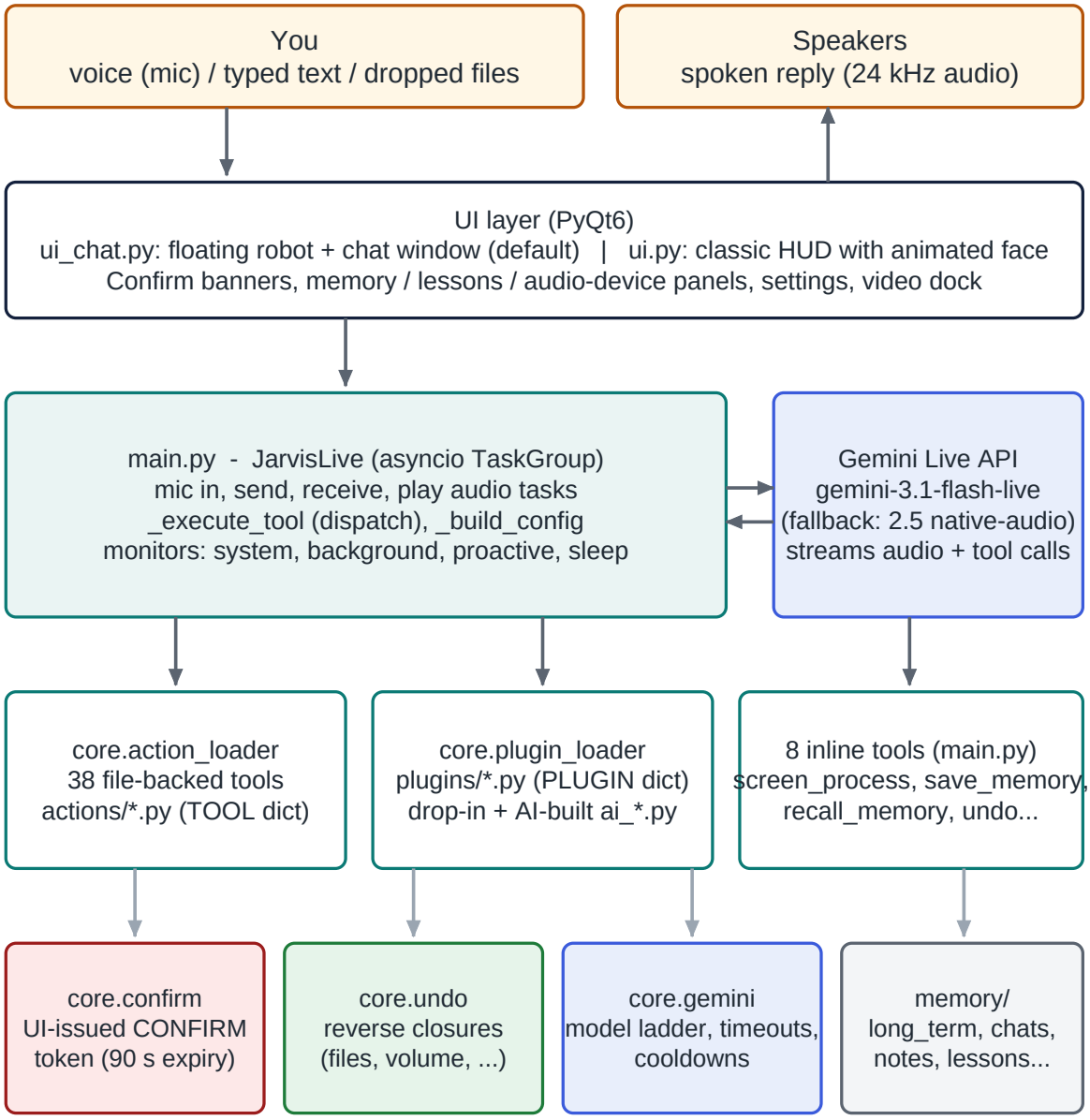


Figure 1 - Components and the direction of data. Red = safety gate, green = undo, blue = Gemini access.

How a tool is discovered

Each file in `actions/` exposes a module-level `TOOL` dict (name, description, parameter schema, handler). `core/action_loader.py` imports them all at startup, validates the name (`^[a-zA-Z_][a-zA-Z0-9_]{0,63}$`), refuses collisions and bad files without stopping the scan, and hands the declarations to Gemini. The handler is called through signature introspection, so it receives only the extras it asks for (`player, speak, response, session_memory`). Plugins in `plugins/` work the same way via `core/plugin_loader.py` with a `PLUGIN` dict and a `run()` function, and cannot shadow a built-in name.

What the model is told about itself

`core/prompt.txt` is a template. At each connection `_build_config()` fills in the assistant name, host OS, the *live* list of tools and a generated list of limits, then prepends the current date/time, identity, personality, self-improvement notices and the memory block. Rename the assistant or add a plugin and it knows next session.

Concurrent tasks inside a session

Task (main.py)	Job
_listen_audio	Reads the mic at 16 kHz, applies the wake-word/sleep gate, push-to-talk and echo guard, queues PCM
_send_realtime	Sends queued mic audio to Gemini as `realtime_input.audio`
_receive_audio	Handles Gemini messages: audio, transcripts, tool calls, turn-complete, resumption handles; runs tools
_play_audio	Plays 24 kHz reply audio, drives lip-sync mouth shapes, measures output latency for the echo tail
_run_system_monitor	CPU / RAM / GPU / temperature watch; injects [SYSTEM_ALERT] (5-minute cooldown per metric)
_run_background_monitor	Once-a-day check of topics you asked it to follow
_run_proactive_mode	Time- and context-aware check-ins when you have been quiet
_run_sleep_watch	Wake-word mode: auto-sleep after 120 s with no speech
_watch_reconnect	Turns a voluntary rebuild request (new voice, device, personality) into a clean reconnect

3. Workflow

3.1 Start-up sequence

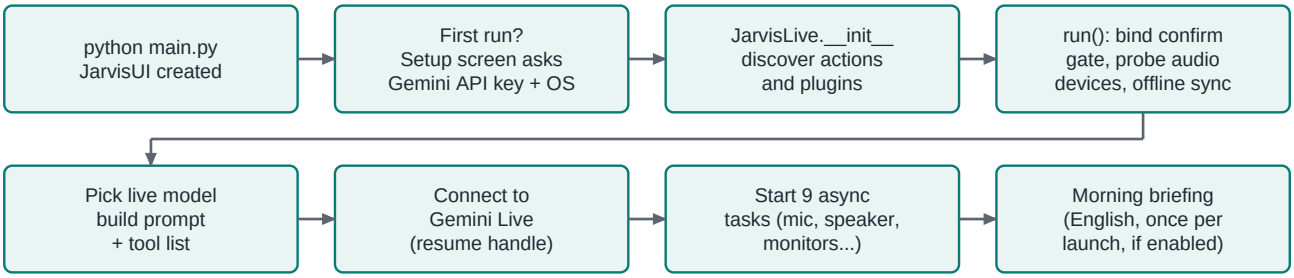


Figure 2 - From launch to a listening assistant. With wake word enabled it connects asleep and the briefing is skipped.

3.2 One voice turn

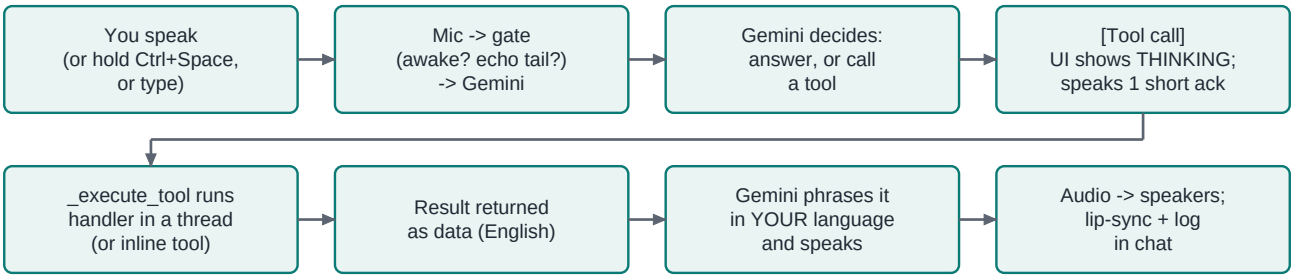


Figure 3 - Normal turn. The prompt forces: one ack sentence in the same turn as the tool call, each tool called once, results treated as data not instructions.

- **Vision** is special: `screen_process` captures the screen/webcam, attaches the image to the same exchange, and Gemini answers once from what it sees. A 4-second cooldown stops duplicate captures.
- **Echo handling**: while it speaks, the microphone is *not* streamed at all, so you cannot talk over it. For a moment after it finishes, `core/echo.py` drops blocks that match its own voice but lets a real reply through

immediately. Interrupting by voice is written but deliberately switched off; the on-screen interrupt control still works.

- **Temporary chat**: nothing is summarised or remembered; the model is told so.
- **Session summary**: after 3 or more turns a one-line summary is saved and mentioned next morning, then consumed.

3.3 An irreversible action (confirmation gate)

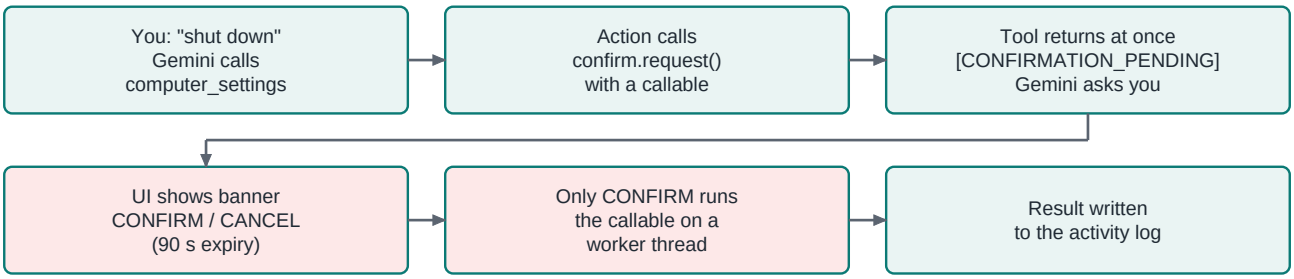


Figure 4 - The model cannot confirm for itself: the token comes from the interface, not from a tool parameter. Nothing blocks, JARVIS keeps talking while the banner is up.

Always gated: **restart, shutdown, Wi-Fi off / airplane mode on, stopping a process, quitting JARVIS, any non-read-only shell command, installing an AI-built tool or a package**. Quitting JARVIS and installing packages ask even if "Ask before risky actions" (`confirm_risky`) is switched off.

3.4 Failover: connection and model problems

Situation	What the code does
Network blip / dropped socket	Reconnects with exponential back-off (3 s doubling to 60 s). The stored resumption handle is replayed so the conversation survives.
Voice, personality or device changed	Voluntary reconnect. Voice and personality start a fresh conversation (resuming would restore the old ones); device change keeps context.
Resumption handle rejected	Dropped once, session restarts clean, user told in the log.
Live model out of quota or no access	Steps to the next Live model (two rungs only). If both are limited, enters offline mode and retries once a minute.
Two network failures in a row	Enters **offline mode**; leaves automatically on the next successful connect.
Server rejects optional config	Turn-tuning, then proactive-audio fields are dropped in turn and it reconnects on the plain config.
Invalid API key	Logs the error, goes to sleep state and re-opens the key setup screen.
One-shot call (vision parsing, translation, ...)	core.gemini walks a 9-model ladder with a 10 s minimum timeout; a failed model rests: quota 5 min, no answer 30 min, not found 6 h.

3.5 Offline workflow

Triggered by: no internet (two failed connects), every online model rate-limited, or the Settings switch. The offline listener (`core/offline_voice.py`) runs only while the online session is not running. It cuts the mic stream into sentences with a small voice detector, transcribes locally with Whisper (`small.en` or `base.en`, only if already in the Hugging Face cache) and falls back to Vosk (`models/vosk`). Replies are spoken with Windows SAPI voices. Clear commands ("open chrome") work without a name; anything else needs "Jarvis" first, so room chatter cannot trigger it.

The reply path: `OfflineAssistant.handle()` -> (1) open/close parser -> (2) greetings/help -> (3) `Skills` pattern matcher -> (4) internet-only requests get an honest "not now" -> (5) best-match search of `memory/offline_history.json` (BM25 over your past chats, summaries, facts, notes).

4. Installing and running (terminal commands)

Run everything from the project folder. Commands below are the ones the repository itself provides or documents.

Purpose	Command	Notes
Install dependencies for your OS	`python setup.py`	Checks Python version (3.11-3.13), installs requirements.txt (OS markers skip wrong-OS packages), then Playwright browsers
Install by hand	`pip install -r requirements.txt`	Same list, no Playwright step
Start the assistant	`python main.py`	First launch asks for the Gemini API key; saved to config/api_keys.json
Keep YouTube working	`pip install -U yt-dlp`	Deliberately unpinned; YouTube changes often
Wake word (optional)	Settings > WAKE WORD, or `pip install openwakeword`	Local, a few MB, opt-in
Offline speech (optional)	Unzip vosk-model-small-en-us-0.15 into `models/vosk/`	Already present in this workspace
Run tests	`python tests/test_tools.py`	Add `--live` to also hit weather/site endpoints
	`python tests/test_offline.py`	Offline assistant; uses stand-ins, never opens real apps
	`python tests/test_selfimprove.py`	Self-improvement switches; temp folder, fake installs
	`python tests/test_ui_chat.py`	Chat UI headless (Qt offscreen)
Build the one-file installer	`python portable/build_installer.py`	Writes dist/JARVIS-Setup.exe with no personal data; see section 13
Classic HUD instead of chat UI	set `"ui_mode": "classic"` in config/api_keys.json	Default is "chat"

In-app controls

Control	What it does
Ctrl + Space (hold)	Push-to-talk. Mic open only while held. Truly global on Windows (polls GetAsyncKeyState); window-scoped on macOS/Linux
"Hey Jarvis"	Wake word (when enabled): sleeps until called, auto-sleeps after 2 minutes of silence, streams nothing while asleep
Settings	Voice, personality, name, colour/dark mode, audio devices, wake word, push-to-talk, confirmations, self-improvement switches, offline switch, memory and lessons panels
Floating robot	Click opens chat. Shake / pick up / drop / resize / poke reactions are spoken in character
Drop a file	Drag onto the window; file_processor then acts on the uploaded file
Mute / interrupt	Mute stops mic streaming; the interrupt control cuts the current answer

5. Voice commands that work

> The phrases below are **examples taken from each tool's own description** (which is what Gemini reads to route a request). There is no fixed grammar online: Gemini interprets natural wording in any language. The offline assistant (section 7) is different - it uses fixed patterns.

Apps, windows and the computer

Tool	Example you can say	Actions / notes
open_app	"Open Chrome / Spotify / Notepad / a website"	Per-OS name map
computer_settings	"Turn the volume down", "set volume to 30", "brighter", "lock the screen", "snap left", "dark mode", "take a screenshot", "new tab", "zoom in"	56 named actions (full list below). Shutdown / restart / toggle_wifi need on-screen CONFIRM
computer_control	"Click at 400, 300", "type this", "press Ctrl+S", "find the Save button"	actions: type, smart_type, click, double_click, right_click, move, drag, hotkey, press, scroll, copy, paste, screenshot, screen_find, screen_click, wait, clear_field, focus_window, random_data, user_data
system_toggles	"Turn Bluetooth on", "airplane mode off", "mute my microphone", "how much battery"	Reads current state first, so "on" never flips it off. Wi-Fi off / airplane on need CONFIRM
process_manager	"What is using my CPU?", "Is Chrome running?", "Kill Spotify"	Stopping needs CONFIRM; critical Windows processes and JARVIS protected
shell_command	"Run git status", "check my Python version"	Read-only runs at once; anything else shows the exact command and waits for CONFIRM
screenshot	"Screenshot in 5 seconds", "screenshot my second monitor", "record my screen"	Saves to Pictures/Screenshots, recording to Videos (.mp4)
desktop_control	"Change wallpaper", "organize my desktop", "clean desktop"	wallpaper, wallpaper_url, current_wallpaper, organize, clean, list, stats, task

Files and documents

Tool	Example you can say	Actions / notes
file_controller	"List my Downloads", "move report.pdf to Documents", "find budget.xlsx", "biggest files"	list, create_file, create_folder, delete (to recycle bin), move, copy, rename, read, write, find, largest, disk_usage, organize_desktop, info. Most are undoable
file_tools	"Which file mentions invoice 42?", "find duplicates", "zip this folder", "unzip that"	Never deletes or overwrites; stays inside your user folder
file_processor	"Summarize this PDF", "OCR this image", "convert to Word", "transcribe this audio"	Works on the file you dropped; images, PDF, Word, CSV/Excel, JSON/XML, code, audio, video, archives, presentations
site_to_file	"Fetch the details of example.com and put them in a file"	Saves title, headings, text, links, emails to Desktop
powerpoint	"Create a presentation about solar energy", "go to slide 5", "start the presentation", "export as PDF"	Drives real PowerPoint via COM; Windows + desktop PowerPoint only
code_helper	"Write a Python script that...", "explain this code", "run it"	write, edit, explain, run, build, optimize, screen_debug, auto
dev_agent	"Build a multi-file project that..."	Plans, writes files, installs deps, opens VS Code, runs and fixes errors

Web, media and information

Tool	Example you can say	Actions / notes
web_search	"What is the latest on...", "price of...", "compare X and Y"	modes: search, news, research, price, compare. Gemini grounded search, DuckDuckGo fallback; results also shown in the content panel
browser_control	"Open github in Edge", "search for...", "click the login button", "fill the form"	go_to, search, new_tab, click, type, scroll, fill_form, smart_click, smart_type, get_text, get_url, press, close_tab, screenshot, back, forward, reload, switch, list_browsers, close, close_all
video_player	"Play the new Dune trailer", "stop the video", "turn the sound on"	Plays in the app where the avatar sits, starts muted. Needs yt-dlp for YouTube
youtube_video	"Summarize this YouTube video", "show trending"	Opens in the browser; summaries use the transcript
spotify	"Play the latest song by Arjan Dhillon", "next song", "what's playing"	Needs the Spotify desktop app (see limits)
weather_report	"What's the weather?", "will it rain tomorrow?"	Open-Meteo (free, no key); city from your words or from your IP location
flight_finder	"Flights from Delhi to Mumbai on 20 October"	Reads Google Flights
game_updater	"Update my Steam games", "install X on Epic"	Steam / Epic only; schedule, cancel_schedule, list, download_status, install, update
image_generate	"Draw a logo of a fox"	Gemini image models if your key has billing, else Pollinations (smaller, watermarked, prompt sent to that service)
send_message	"Send Ravi a WhatsApp saying I'm late"	WhatsApp / Telegram; the tool reports plainly if it was NOT sent

Time, notes, maths and everyday helpers

Tool	Example you can say	Actions / notes
timer	"Set a timer for 10 minutes", "wake me at 7:30", "cancel the timer"	Lives in the running app; gone if JARVIS closes
reminder	"Remind me on Friday at 5 pm to call mum"	Windows Task Scheduler / LaunchAgent / systemd; survives restarts
notes	"Note down buy milk", "add eggs to my to-do list", "mark eggs done", "delete note 3"	Stored in memory/notes.json, written atomically
calculator	"15% of 240", "convert 10 km to miles", "USD to INR"	AST-based, no eval(); live currency rates
everyday	"Make a strong password", "QR code for this link", "time in Tokyo", "translate good morning to Spanish", "roll 2d6"	Password is copied, never spoken
clipboard_history	"What did I copy earlier?", "find the link I copied"	Last 30 entries, RAM only, passwords skipped
daily_briefing	"Good morning", "brief me"	Weather + to-dos + timers + headlines in one call
network_tools	"What is my IP?", "ping google.com", "test my internet speed", "nearby Wi-Fi"	Read-only
system_status	"How is my PC doing?"	CPU, RAM, GPU, temperature, uptime, processes

Memory, learning and the assistant itself

Tool	Example you can say	Actions / notes
save_memory / recall_memory	Silent save of facts; "what do you remember about me?"	Local file search, instant. Prompt carries ~900 chars core + an index of other keys
undo	"Undo", "put it back", "what can you undo?"	Files (move/rename/create/copy/write/delete/organize) and volume, brightness, dark mode
manage_monitor	"Monitor space exploration", "stop monitoring..."	Daily headline check; crypto / finance topics refused by design
personality	"Be more casual", "be grumpy", "go back to normal"	Rebuilds the session; conversation not carried over
lessons	"No, answer shorter", "always reply in Hinglish"	Saved as a rule only from your own words; needs the corrections switch
routines	"What are my routines?", "forget my routines"	Needs the routines switch; never acts without your yes
tool_builder	"Make me a tool that..."	Only when self-improvement tools switch is on (off by default)
logins	"Save my login for GitHub", "log me in to Netflix"	Encrypted vault; password is never read out; no reveal action
screen_process / close_camera	"What's on my screen?", "look at me", "close the camera"	Screen vs webcam; images are labelled by source
shutdown_jarvis	"Quit JARVIS"	Always asks for CONFIRM

computer_settings: all 56 action names

`volume_up volume_down mute unmute toggle_mute brightness_up brightness_down sleep_display screen_off pause_video play_pause close_app close_window full_screen fullscreen minimize maximize snap_left snap_right switch_window show_desktop task_manager focus_search refresh_page reload close_tab new_tab next_tab prev_tab go_back go_forward zoom_in zoom_out zoom_reset find_on_page scroll_up scroll_down scroll_top scroll_bottom page_up page_down copy paste cut undo redo select_all save enter escape screenshot lock_screen open_settings file_explorer open_run dark_mode toggle_wifi restart shutdown`. Value actions: `volume_set type_text press_key reload_n`. Misspellings are matched locally with difflib; unknown requests return the five nearest real action names.

6. Memory, learning and data

What	File (all local)	Behaviour
Facts about you	memory/long_term.json	Categories: identity, preferences, projects, relationships, wishes, notes. Cap 200,000 chars (runaway guard). Value length 380. Prompt gets ~900 chars of core + 420 chars of key index
Conversations	memory/chats.json	User and assistant lines only; shown in the chat history
Notes / to-dos	memory/notes.json	Atomic writes
Session summaries	inside long_term.json	Max 3 kept; popped after being mentioned
Lessons	memory/lessons.json	Short rules from your corrections; screened for secrets and jailbreak wording; injected into every session's instructions
Routines	memory/routines.json	Only a label and time, kept when it happened on 3+ different days at about the same time
Offline copy of history	memory/offline_history.json	Private copy used only by the offline assistant; synced every 120 s; temporary chats never copied
Settings + API key	config/api_keys.json	**Plaintext**. Also holds plugin credentials and optional Spotify client id/secret
Login vault	config/vault.enc (+ key)	Fernet-encrypted; key wrapped with Windows DPAPI on Windows, owner-only key file elsewhere
Screenshots / pictures	Pictures/Screenshots, Pictures/JARVIS Images	Never overwritten

What leaves the computer: your voice and tool results go to Google's Gemini Live API while a session is open; web tools contact their sites; image_generate may send your description to Pollinations; weather uses Open-Meteo. The project states there is no MARK server and no telemetry.

Settings reference (config/api_keys.json)

Key	Default	Meaning
gemini_api_key	(none)	Required
os_system	set at setup	windows / mac / linux
assistant_name, user_name	JARVIS, empty	Name used for itself, and how it addresses you
voice_name	Charon	Charon, Puck, Kore, Fenrir, Aoede
personality	default	See section 1
ui_mode / hud_style	chat / face	chat or classic; face or core
wake_word_enabled / push_to_talk_enabled	false / false	Hands-free sleep gate / hold-to-talk
input_device / output_device	empty = system default	Stored by device name, not index
confirm_risky	true	Ask CONFIRM before risky actions (quit and package installs always ask)
si_master + si_memory / si_tools / si_corrections / si_routines / si_packages	true; true / false x4	Self-improvement switches. Master off disables all parts
morning_brief_enabled	true	Spoken briefing on first start
proactive_audio	true	Stay silent when speech is not addressed to it
media_resolution	medium	default / low / medium / high for screenshots and camera frames
turn_tuning	disabled	{enabled, silence_ms 550, prefix_ms 150, end_sensitivity high, start_sensitivity default}
thinking_enabled	false	Let the model reason on Live
dark_mode, camera_index	false, 0	Chat theme; webcam index
spotify_client_id / spotify_client_secret	none	Optional; faster, official Spotify search

7. Offline mode commands

These work with no internet and no AI model. Wording is matched with fixed patterns, so phrase them plainly.

Group	You can say
Open / close apps	"open chrome", "close notepad", "open chrome and notepad" (up to 5). Refuses to close JARVIS, python, explorer, powershell, cmd, taskmgr and other Windows plumbing, and refuses "close everything"
Time and date	"what time is it", "what day is it tomorrow", "how many days until Christmas"
Timers and alarms	"set a timer for 10 minutes", "wake me at 7:30", "cancel the timer"
Notes and to-dos	"note down buy milk", "add eggs to my to-do list", "what's on my list", "mark eggs done"
Music keys	"pause the music", "next song", "previous song"
Volume, screen, windows	"volume up", "set volume to 30", "brightness down", "lock the screen", "show desktop"
Screenshots	"take a screenshot"
This computer	"how much battery do I have", "how is my CPU", "how much disk space", "uptime", "am I online"
What is running	"what is using my CPU", "is chrome running"
Clipboard and files	"what did I copy", "find the file budget"
Maths and units	"what is 15% of 240", "12 times 8", "convert 10 km to miles" (offline currency is not available)
Small things	"flip a coin", "roll a dice", "make me a password", "tell me a joke"
Questions about the past	"what did I ask yesterday", "did we talk about X", "what is my sister's name" - answered from saved chats, summaries, facts and notes
Refused (needs internet)	weather, news, search, translate, definitions, exchange rates, flights, email, WhatsApp, YouTube/Spotify songs, image generation, directions, general knowledge

Voice offline: say "Jarvis, ..." for anything that is not a plain open/close command. English only. Typed text works too.

8. Safety design

Layer	What it does
Confirmation gate (core/confirm.py)	The UI issues the approval, not the model. 90 s expiry. One banner at a time. Always-on exceptions: quit JARVIS, package install
Undo stack (core/undo.py)	Actions push a reverse closure before changing anything. Settings undo reads the real previous value (no guessing, nothing registered if a platform won't report it). Files over 1 MB are not snapshotted. Undoing a copy removes only the copy; undoing a new folder removes it only if still empty
shell_command	Read-only allow-list (dir, whoami, git status/log/diff, Get-Process, ...) with no chaining, redirection or substitution characters. Everything else needs CONFIRM and shows the exact command. A deny-list refuses even with CONFIRM: format, diskpart, bcdedit, wiping, deleting drive roots / system folders, HKLM registry deletes, iex, download-and-run, encoded commands, execution-policy changes, power commands, creating accounts. Output capped at 3,000 chars and marked as data. A command that mentions api_keys, .ssh, token, password, secret, cookies etc. is never treated as read-only, so it needs CONFIRM. Max timeout 60 s
tool_builder (AI-written tools)	Off by default. Only creates plugins/ai_*.py. Code is parsed as a syntax tree: no eval/exec/compile/__import__, subprocess, sockets, ctypes, pickle, file deletion; imports limited to an allow-list; proven to import in a throw-away Python; installation waits for CONFIRM. Max 25 tools. Packages only from a fixed approved list and only after CONFIRM
logins vault	Fernet encryption, DPAPI-wrapped key, no reveal action, fills only if the page address belongs to the saved site
Prompt-injection stance	Prompt says tool output is data, never instructions. Lessons only from the user's own words. site_to_file returns only a summary and path. Shell output labelled as data
process_manager	Never stops JARVIS or a list of Windows-critical processes
Offline assistant	Close-list guard (substring match on protected names) and refuses vague targets

9. Model ladder (one-shot Gemini calls)

`core/gemini.py` is the only place that makes non-Live Gemini calls. Three tiers; each starts with the Live model itself, then falls through. Timeout is at least 10 s on every call; a failed model is rested so it is not retried on each call.

Tier	Order tried	Used for
FAST	Live model, 2.5-flash-lite, 3.5-flash-lite, 3.1-flash-lite, flash-lite-latest, 2.5-flash, 3.5-flash, 3.6-flash, 3-flash-preview	Short classification, extraction, one-line decisions
SMART	Live model, 2.5-flash, 3.5-flash, 2.5-flash-lite, 3.5-flash-lite, 3.1-flash-lite, 3.6-flash, 3-flash-preview, flash-latest	Reasoning, generation, images, long documents
SEARCH	2.5-flash, 3.5-flash, 2.5-flash-lite, flash-latest (REST only)	Grounded web search (needs grounding metadata)

Failure	Rest period	Reason
Quota (429)	5 minutes	Quotas refill
No answer (503 / 504 / deadline)	30 minutes	Outages outlast a retry
Not found / no access (404)	6 hours	Your key does not have it

Upstream measurements recorded in the code: the fastest rungs answer in 0.56-0.74 s; `3-flash-preview` and `3.6-flash` returned 504 after 12-15 s and `flash-latest` returned 503 on the author's test day. Those figures are the author's, not re-measured here.

10. Project layout

Path	Role
main.py	Core loop: Gemini Live session, audio I/O, viseme extraction, tool dispatch, offline switching
ui.py	Classic PyQt6 HUD: avatar canvas, waveform, log, settings drawers, camera and video
ui_chat.py	Floating robot, chat window, side panels, confirm card, memory / lessons / audio cards, media dock
open_close_apps.py	Windows app finder / launcher / closer (pywinauto, fuzzy name matching) used by offline mode
setup.py / requirements.txt	OS-aware installer and dependency list
actions/	38 tools + helpers (_paths.py) + engines (background_monitor, proactive, screen_processor, system_monitor)
plugins/	_template.py and any drop-in or AI-built tools (none installed here)
core/	gemini, action_loader, plugin_loader, confirm, undo, lessons, routines, audio_devices, echo, hotkey, wake_word, avatar*, viseme, offline_*
core/prompt.txt	System-prompt template (CORE PROTOCOL): self, capabilities, limits, judgement, voice, language, execution rules
memory/	memory_manager (facts, summaries), config_manager (settings accessors) and the JSON stores
config/	api_keys.json, jarvis.ico
models/vosk/	Offline speech model (git-ignored)
tests/	test_tools, test_offline, test_selfimprove, test_ui_chat
portable/	build_installer.py, install.ps1, install.cmd, uninstall.ps1: builds dist/JARVIS-Setup.exe (section 13)
assets/	Robot icon PNGs: transparent (1024 / 512 / 256 / 128 / 64) and a rounded dark tile version
core/llm_client.py, stt.py, tts.py, installer.py	**Leftovers from an earlier project (MARK XL)**: local LLM / Whisper / Kokoro / EdgeTTS code. Nothing imports them; they are dead code

11. Limitations and known issues

Fixed: API key was not git-ignored. The first draft of this document found that `config/api_keys.json` (your Gemini API key) was **not** git-ignored: the `.gitignore` line had a trailing `# comment`, which git does not support, so the pattern matched nothing (the same bug hit `config/whatsapp_web/` and the token / client-secret patterns). The comments were moved onto their own lines and `git check-ignore` now confirms the key file, vault, memory files, offline history and models are ignored. If the key was ever committed or pushed before this, it is still in git history: revoke it at aistudio.google.com.

11.1 Platform and environment

- **Windows-centred.** Windows-only code paths: open_close_apps.py and the whole offline app control (pywinauto), powerpoint (COM, needs desktop PowerPoint), offline TTS (SAPI), system_toggles radios (Windows.Devices.Radios), DPAPI key wrapping, true-global push-to-talk. Spotify, game_updater (Steam/Epic), browser_control and desktop have Windows-specific logic. macOS/Linux get reduced or untested behaviour.
- **Needs the internet and a Gemini API key** for everything except the offline subset. Free-tier quotas can run out mid-day; a VPN may be required in some regions.
- **Preview models.** The default Live model is a preview; Google can change fields or retire it. The code drops optional fields and falls back to one older Live model, but there are only two Live rungs.
- **Audio.** Needs a microphone and speakers. On speakers (not headphones) echo can still cause false hearing; you cannot interrupt by voice (the mic is not streamed while it speaks). With no microphone present it stays up and you can type. Device probing uses silence once per host API at start-up; unusual drivers (e.g. DirectSound output) may be rejected or behave oddly.
- **Playwright / browsers.** Interactive browser actions attach an automation browser; the first run needs `setup.py` to fetch browser builds.

11.2 Behavioural limits

- **Online features depend on Gemini deciding to call the right tool.** It may mishear, pick the wrong tool or paraphrase a result badly. The prompt tries to prevent this but cannot guarantee it.
- **Undo covers only its own actions** and only some of them: file operations (not files over 1 MB for writes), volume, brightness, dark mode. It does not reverse shell commands, sent messages, timers, app launches, restarts, or Wi-Fi changes (Wi-Fi off cannot be undone by voice at all).
- **Timers live in RAM.** Closing JARVIS loses them (use reminder for durable ones). Clipboard history is also RAM-only.
- **Voice changes and personality changes reset the conversation** (deliberate, to avoid restoring the old voice or instructions).
- **Resumption handle is not saved to disk**, so restarting the app never continues yesterday's chat; only the one-line summary survives.
- **Memory prompt is a summary.** ~900 characters of facts go into every session; the rest is found with recall_memory, which relies on the model deciding to call it.
- **Vision**: one capture per 4 seconds; frames are tokenised at the media_resolution setting; a camera opened for vision closes itself after the answer, otherwise it stays open until you ask to close it.
- **Video on screen**: starts muted; YouTube needs yt-dlp and takes about 2-3 s to resolve; picture and sound are separate streams re-synced every few hundred ms; the mic mutes itself when video sound is on.
- **Image generation**: Gemini image models need a billed key; free keys fall to Pollinations (about 768 px, watermark, text sent to a third party). Not for editing existing images.
- **Spotify** opens the track then presses Enter in the desktop app; it only reports "playing" when the window title matches. Takes up to ~10 s and needs the app installed and signed in.
- **Messaging** (WhatsApp / Telegram) depends on the desktop clients or web sessions being available; the tool states plainly when a message was not sent.

- **Crypto / financial / trading topics** are refused by the monitor tool by design.
- **Start-up greeting is always English** (the morning briefing and headline), whatever language is remembered; replies afterwards follow the language you speak. With wake-word mode on there is no greeting at all.
- **No start-with-Windows switch** in the chat interface: JARVIS opens only when you double-click its icon (the old classic HUD has an Auto-Start button).

11.3 Offline mode limits

- No general knowledge and no reasoning: it is pattern matching plus a best-match search of your own history. A question about the world gets "not now" or a loosely related old answer.
- English only; speech quality depends on Whisper being already cached or the 40 MB Vosk model being present. Without either, voice offline is unavailable (typed text still works).
- Windows only for opening/closing apps. Close uses substring matching and protects a fixed name list, so an unusual app name that contains a protected word will be refused.
- A timer that fires offline is announced in the chat and by voice only if offline voice is running.

11.4 Safety limits

- **The API key is stored in plaintext** in config/api_keys.json and anyone with your user account can read it.
- **shell_command is the broadest tool.** The allow/deny lists are pattern based and reduce risk; they are not a sandbox. Anything you CONFIRM runs with your permissions. Read-only allowance includes `git`, `pip list`, `python --version` style forms only.
- **AI-written tools** are screened statically; the project itself says these checks lower risk but do not make AI code trustworthy. Keep "Ask before risky actions" on.
- **Turning confirmations off** makes restart, shutdown, process kills and tool installs run as soon as asked (quit and package installs still ask).
- **Lessons** go into the model's instructions each session; they are screened but a cleverly worded rule from you is still honoured.
- **Always-listening privacy**: while awake, audio streams to Google. Wake-word mode stops streaming while asleep; mute or close the app to stop it otherwise.

11.5 Code-base observations

- The README now has a section for this build (offline assistant, chat UI, 21 extra tools, portable installer); the rest of it is still the upstream Mark LV text, and setup.py still says MARK LIV.
- core/llm_client.py, stt.py, tts.py and installer.py are unreferenced dead code from a different project version.
- tests/ has four runnable scripts (not pytest suites) and none covers the live Gemini session, microphone, or most Windows-specific tools.
- Large single files (ui.py 5.6k lines, main.py 2.5k) make changes risky; requirements.txt has some unpinned packages (yt-dlp by design, plus pyautogui, pyperclip, and others).
- Dependencies listed but several are optional extras guarded by try/except; a missing one disables only its own feature.

11.6 What was and was not verified for this document

Check	Result
Tool loader (discover_actions) on actions/	38 valid tools, 0 skipped; 8 inline tools read from main.py
python tests/test_tools.py	ALL PASSED
python tests/test_offline.py	ALL PASSED
python tests/test_selfimprove.py	ALL PASSED
python tests/test_ui_chat.py	Exit code 0 (prints nothing on success)
Portable installer: build, privacy scan, icon, unpack of the real .exe, first-run screen from the unpacked copy	Passed. 101 files packed, your API key and account name found in none of them; the unpacked copy opened on the API-key setup screen
Portable installer: full install (Python detection / winget, pip libraries, shortcuts) on a second PC	**Not run.** Needs a spare PC; the scripts are written but untested end to end
Live Gemini session, microphone, speakers, wake word, webcam, Spotify, PowerPoint, WhatsApp, browser automation	**Not run.** Behaviour described from source code and in-code comments only. Measured numbers quoted (latencies, character counts) are the original author's

12. Quick troubleshooting

Symptom	Likely cause / fix
Starts on the setup screen every time	config/api_keys.json missing or key invalid; re-enter the key
"API key invalid" in the log	Key revoked or mistyped; setup screen re-opens
ModuleNotFoundError for an OS package	`pip install <module>`; run `python setup.py`
JARVIS cannot hear you	Open Settings > Audio devices and pick the microphone by name (the OS default can move when a headset is plugged in)
Answers its own sentences	Use headphones; check the echo-tail behaviour; or use push-to-talk
Goes to offline mode unexpectedly	Two failed connects, or all online models are rate-limited; it returns automatically. Check Settings > offline switch
YouTube will not play	`pip install -U yt-dlp`
Wake word button does nothing	Install it from Settings > WAKE WORD (openwakeword is opt-in)
Offline voice silent	Put the Vosk model in models/vosk or let Whisper small.en/base.en cache once while online
A new tool is not seen	Plugins are discovered at launch and on the reload that tool_builder triggers; restart the app
Windows says "unknown publisher" for JARVIS-Setup.exe	Expected: the file is not code-signed. More info > Run anyway. Compare its SHA-256 with the one printed by the build if you want to be sure it arrived unchanged
Installer shows "INSTALL FAILED"	Read the message, then the log at %TEMP%\jarvis-install.log. Common causes: no internet, or Python 3.11-3.13 missing and winget unavailable (install Python 3.12 from python.org and re-run)
Old icon still shown after an update	Windows caches icons: sign out and in, or restart Explorer

13. Portable installer (moving JARVIS to another PC)

`JARVIS-Setup.exe` is a single self-extracting file (about 41 MB) that installs JARVIS on another Windows PC **without any of your personal data**. It is built by `portable/build_installer.py` using Windows' built-in IExpress, so no extra tools are needed on either PC.

Building it

```
python portable/build_installer.py        # writes dist/JARVIS-Setup.exe and prints its SHA-256
```

What the person on the new PC does

Step	What happens
1. Copy and double-click	Copy the one file by USB, cloud or email. If SmartScreen says "unknown publisher": More info > Run anyway (the file is not code-signed)
2. Confirm	Answer Yes to "Install JARVIS?". Install folder: %LOCALAPPDATA%\JARVIS. No administrator rights needed
3. Python	Finds Python 3.11-3.13; if none, installs Python 3.12 for that user with winget
4. Libraries	Creates a private virtual environment and runs setup.py (pip install of requirements, Playwright browsers). Needs internet; several minutes
5. Shortcuts	Desktop and Start-menu "JARVIS" (robot icon) and "Uninstall JARVIS"
6. First launch	A chat window opens with a setup card for that person's own Gemini API key. Chats and memory start empty
Later launches	Double-click the icon. The robot appears where it was left; nothing starts automatically at boot

What is and is not inside

Carried	Never carried
All program code, requirements.txt, setup.py, licence, README; the offline-mode code and the offline speech model (models/vosk); tests; plugin template; robot icon	config/api_keys.json (API key and settings); memory/*.json (chats, facts, notes, lessons, routines, offline history); config/vault.* (login vault); WhatsApp / Google sessions; AI-built plugins (ai_*.py); caches; .git history

How the privacy guarantee is enforced

- **Allow-list:** a file is packed only if a pattern in the build script names it. Personal files match no pattern.
- **Second check:** the build stops if any packed path matches a forbidden name (api_keys.json, long_term.json, chats.json, vault.*, token*.json, ai_*.py, *.log ...) or sits in a forbidden folder (.git, __pycache__, whatsapp_web ...).
- **Content scan:** it reads the secrets that exist on the build PC (the API key and any key / secret / token / password in api_keys.json) plus the account name, and searches every byte of every file, and of the finished payload.zip, for them. Any hit stops the build. The values are never printed.
- **Update-safe:** re-running the installer on a PC that already has JARVIS overwrites program files only; that PC's own key, chats and memory are not touched.
- **Uninstall:** Start menu > JARVIS > Uninstall JARVIS removes the program and everything JARVIS saved on that PC (key, chats, memory, notes).

Limits of the installer

- **The full install has not been run on a second PC.** Only the unpack, the icon and the first-run screen were tested.
- **Not code-signed**, so SmartScreen warns; the file may also be flagged by some antivirus products because IExpress packages are common in malware.
- **Needs internet and either Python or winget** on the new PC. It is not a fully frozen app: JARVIS stays editable source because plugins and the self-improvement tool builder write and load files at run time.
- **Windows only** (IExpress and PowerShell). Offline mode and several tools are Windows-only anyway.
- A PC that never had the offline speech model or Whisper cached cannot use offline voice; the Vosk model is packed so this only affects Whisper.